



OpenHarness

Agentic Capabilities & Orchestration

A detailed implementation guide

Agent loops · Patterns · Memory · Experiments

Scalability · Resilience · Human collaboration

REPOSITORY REVIEW / VERSION 0.2.0

27 September 2026 • Commit 30b7f62

[deepfinery/OpenHarness](#)

Source-grounded descriptions, worked examples and implementation limits

Contents

A reference for engineers, architects and operators. Chapters 4–8 explain agent execution; chapters 9–11 cover memory and experiments; chapters 15–17 cover recovery and scale.

1. Purpose and architectural identity	3
2. Capability inventory	5
3. From request to durable execution	6
4. The inner agent loop	8
5. Effort, token accounting and termination	10
6. The four built-in agent patterns	11
7. Harness graphs and composed patterns	12
8. Dynamic subagents and bounded delegation	14
9. Context engineering and evidence preservation	15
10. Memory is several systems, not one cache	16
11. How experiments and learning are stored	18
12. Models, tools, skills and execution context	20
13. Human-in-the-loop orchestration	21
14. Guardrails, hooks and isolation	22
15. Resilience and recovery guarantees	23
16. Scalability and capacity planning	24
17. Machines and fleet-scale agent orchestration	26
18. Studio, APIs, schedules and integrations	27
19. Storage, retention and observability reference	29
20. Worked agentic scenarios	30
21. Verification evidence and practical limits	32
22. Source register	33

1. Purpose and architectural identity

OpenHarness is a self-hosted agent execution system and visual studio. Its central responsibility is to turn a user request into controlled, observable, recoverable model-and-tool work. It combines an agent runtime, a workflow interpreter, persistent memory, MCP connectivity, human intervention, safety controls, and a durable job dispatcher. The repository calls the product OpenHarness; “orchestrator” is the repository and architectural role discussed in this guide.

This document describes the implementation at commit [30b7f628261bf5516c3b3e55871baeb3c80b3973](#), reviewed on 27 September 2026. Package version: 0.2.0. Source code takes precedence where older guides differ. Examples and capacity calculations are explanatory, not measured performance results. This review inspected source and repository tests; it did not benchmark a live deployment or execute real machine remediation.

1.1 The three orchestration levels

The first level is the agent pass: the model receives instructions, context and tools, then repeatedly chooses an action or an answer. The second level is the agent pattern: ReAct, plan-and-execute, reflection or an autonomous loop arranges one or more passes. The third level is the harness graph: nodes route work among agents, explicit tools, parallel groups, human reviews, conditions, email and final output. Dynamic delegation can create child agents inside an agent pass. Scheduled fleet monitoring adds a separate durable coordinator above individual node-scoped runs.

These levels compose. A research node can use plan-and-execute; a downstream reviewer can use reflection; a coordinator can delegate independent research tasks; a condition can route an unsatisfactory result back through a bounded graph cycle. The graph determines who runs next. The pattern determines how one agent approaches its assigned task. Neither should be confused with the RabbitMQ job queue, which distributes accepted top-level jobs to workers.

1.2 Main components

Component	Responsibility	Main implementation
Studio	Harness canvas, configuration, Playground, history, memory, approvals, knowledge and infrastructure views	apps/studio
API and dispatcher	Authentication, resource management, submissions, schedules, events, recovery and background dispatch	apps/api
Runner	Job consumption, run leases, model execution, tool execution, indexing and lesson generation	apps/runner
Shared core	Schemas, patterns, runtime, memory, recovery, safety and integration logic	packages/core
MongoDB	Authoritative execution state and application metadata	Runs, notes, plans, approvals, journals and resource records
RabbitMQ	Persistent delivery of run, index, delete and reflection jobs	agentic.jobs , dead-letter queue

Component	Responsibility	Main implementation
Shared file storage	Uploaded document bodies, continuations, durable tool results and artifact bytes	DATA_DIR/files
Vector store	Search indexes and semantic or hybrid retrieval	Weaviate by default; other drivers available
Device gateway	MCP bridge to devices that connect outward over WebSocket	gateway, connector packages

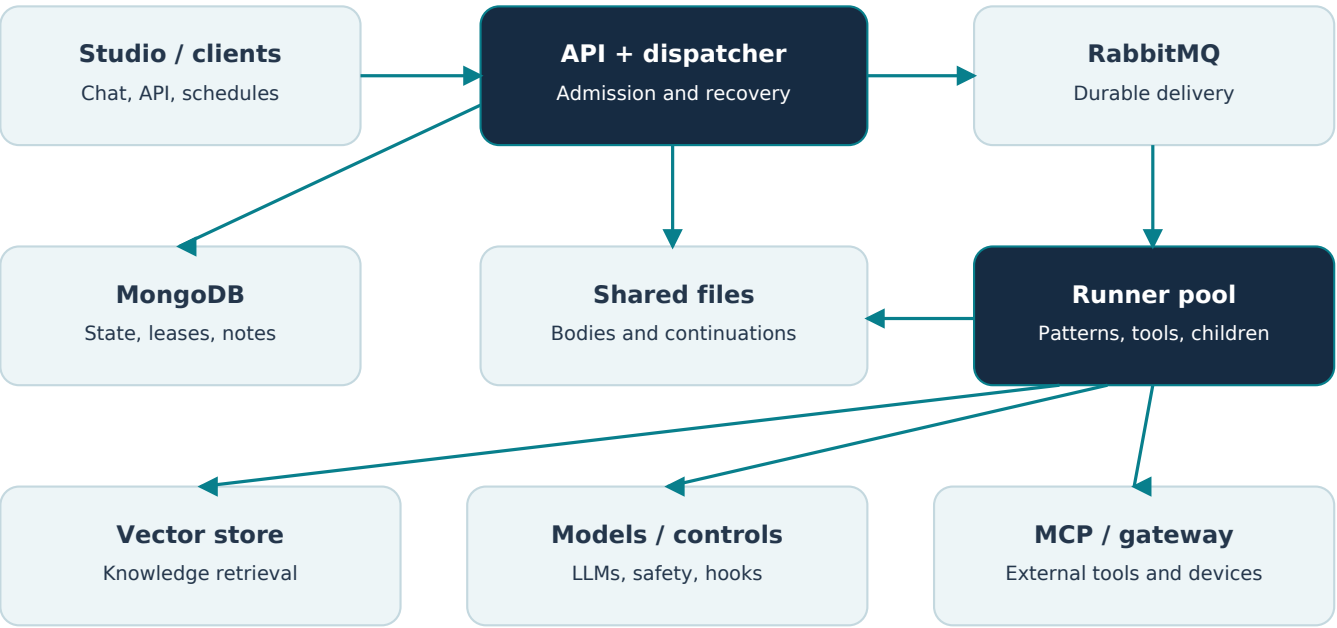


Figure 1. System architecture

1.3 Design boundaries

All external capabilities exposed to agents use MCP. Built-in memory, skill, workspace-file, human-input and delegation functions are runtime facilities. Internal services such as model APIs, SMTP, safety checks and signed webhooks use their own service interfaces; the MCP rule does not mean every internal network request is MCP.

The default deployment is a complete single-host Compose stack. It supports adding runners, but its bundled database, broker and vector store are not a replicated high-availability installation. The repository includes deployment assets and connector source; it does not require a sibling project. There are no provider-specific tool catalogs, federated user-login flows, billing systems or customer-management modules.

Source basis: [S01], [S02], [S03], [S04].

2. Capability inventory

The following inventory groups the product surface by the work it enables. Later chapters explain the agentic mechanisms and their limits.

Area	Implemented capabilities
Agent configuration	Instructions, model provider, timezone, tool allow-lists, knowledge attachments, notebooks, skills, safety policies, effort, explicit limits and optional delegation
Agent patterns	ReAct; plan-and-execute; draft/critique/revise reflection with optional judge model; bounded autonomous iteration
Harness design	Agent cards, Start/Finish, resource attachments, conditional branches, parallel groups, explicit MCP steps, review nodes, email, bounded cycles, YAML, undo/redo
Starter recipes	Knowledge research, MCP assistant, research/review, planner/researcher/writer, specialist routing, research/email, machine operator and blank canvas
Execution	Durable submissions, configuration snapshots, step outputs, continuations, tool journaling, streaming, cancellation and recovery policies
Delegation	Model-selected focused child tasks, fresh contexts, shared task notebook, bounded fanout, budget allocation and parent/child traces
Context management	Prompt-size estimation, provider context-error recovery, conversation compaction, tool-result offloading and final-answer reserves
Temporary memory	Query-scoped Markdown notes, immediate literal search, paged reads, source references, seven-day retention and selective promotion
Long-term knowledge	Markdown notebooks, document upload/indexing, citations, vector/hybrid search, recent-note fallback and multiple attached destinations
Experiments and learning	Automatic unreviewed experiment records, feedback, failure lessons, scoped recall and workflow JSONL export
Additional persistent state	Agent core-memory blocks, archives, harness workspace files, execution plans and skill versions
Human collaboration	Questions, risk-based or explicit approvals, editable arguments, review branches, Inbox, expiry/escalation and optional email notification
Safety	Additive policy attachments, five inspection stages, allow/block/redact, built-in and NeMo checks, semantic classifiers, policy YAML and bounded evaluations
Models	OpenAI-compatible endpoints, Anthropic, Gemini and Ollama adapters; separate chat/embedding setup; model testing and per-execution selection
MCP	HTTP/SSE connections, discovery, schema validation, token/OAuth authentication, resource/prompt access and explicit tool permissions
Machines	Outbound connectors, inventory, device tool policies, gateway auditing, approval integration and selected-machine execution
GPU clusters	Shared enrollment, installation recipes, durable monitoring waves, typed GPU diagnostics and controlled remediation

Area	Implemented capabilities
Entry points	Playground, API, durable conversations/sessions, authenticated webhooks, schedules and revocable embedded chat
Integrations	Open Harness API adapter, generated API explorer, OpenAPI download, JavaScript/TypeScript client, signed event webhooks and streams
Workspace administration	Local users, administrator-managed teams, tenant isolation, scoped keys, resource ownership and edit-conflict detection
Observability	Run history, bounded trace, token usage, saved reflection stages, memory status, child runs, artifacts, diagnostics and safety audit
Operations	Compose install, health checks, persistence, runner scaling, shared-filesystem configuration and isolated integration-test assets

“Implemented” here means a repository capability exists. It does not imply unlimited scale, universal provider compatibility or a guarantee of answer correctness. For example, a successful reflection pass records a model-authored critique; it is not proof that every factual claim was independently verified.

Source basis: [S01], [S05], [S06], [S21], [S23], [S24], [S25].

3. From request to durable execution

3.1 Admission and snapshot construction

A request identifies a harness or supported agent target and supplies its input, optional conversation context, payload and execution options. The API verifies tenant ownership, enabled resources and the caller's scope. An idempotency key is scoped to the owner. Repeating the same request returns the existing run; reusing its key with a different request hash returns a conflict. Scheduled submissions also have deterministic keys.

Before inserting a new run, admission counts the owner's queued, running and waiting-for-human runs against `MAX_ACTIVE_RUNS`. The default is 20. This is application admission control, implemented as a count followed by insertion, rather than a globally atomic capacity reservation. Simultaneous submissions can therefore require additional operational headroom.

The accepted record snapshots the workflow, compiled agent configurations, resolved skill contents and guardrail policy revisions. Resource attachments are compiled into agent settings. Per-run overrides and selected-machine bindings are applied to that snapshot. A later canvas edit does not rewrite the accepted definition.

Snapshotting is deliberately narrower than a complete environment freeze. Provider records, MCP connections, external data, mutable knowledge and execution-time hooks are not all copied into one immutable execution bundle. The runtime resolves provider and connection records as it executes. A stored run is strong provenance, but exact reproduction also requires the same external models, tool behavior and evidence.

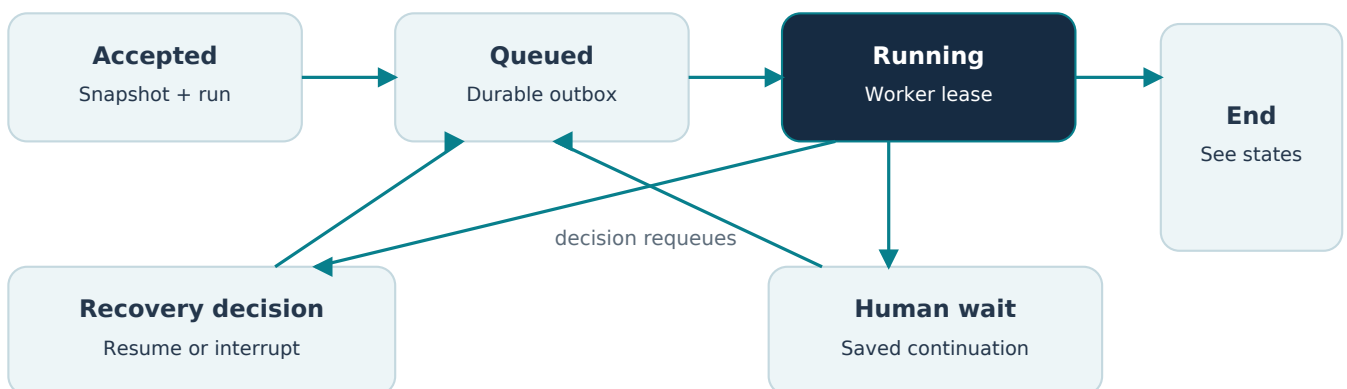
3.2 Queue delivery and worker ownership

MongoDB insertion precedes queue publication. The run document is the durable outbox: failure to reach RabbitMQ does not erase an accepted request. A dispatcher retries pending publication. RabbitMQ uses a durable queue, persistent messages, publisher confirms and a dead-letter destination.

A runner claims work with an atomic transition from queued to running and assigns a lease ID. A duplicate queue message cannot claim a run that is already active. The worker lease lasts 30 seconds and is renewed every five seconds. State writes use the current lease ID so a displaced runner cannot continue updating the run as its owner.

3.3 Execution states

State	Meaning and next possibilities
queued	Persisted and awaiting a runner; may be a new submission or a recovered continuation
running	Claimed under a lease; executing the graph or one agent
waiting_for_human	Continuation saved; worker released; awaiting a durable question or approval decision
succeeded	Runtime completed and produced an available final answer; content remains subject to evaluation
failed	Execution error or unavailable final synthesis; partial evidence may remain readable
cancelled	Explicit cancellation; pending human work and child execution are also addressed
interrupted	Work could not safely resume, or recovery policy/limits prevented continuation



Terminal outcomes: succeeded, failed, cancelled or interrupted

Figure 2. Execution lifecycle

The worker streams model text through a bounded persisted buffer, saves final output and attempts to create `answer.md`. Terminal handling settles conversation state, saves eligible experiments, emits events and requests failure learning where configured. If artifact creation fails, the run result can still contain the answer. If final synthesis itself is unavailable, the run is failed rather than being reported as successful solely because some earlier work completed.

Source basis: [S03], [S04], [S07], [S08], [S09].

4. The inner agent loop

Every built-in pattern ultimately uses the same bounded conversation engine. This gives different reasoning arrangements a common implementation for model access, tools, memory, approvals, context control and recovery.

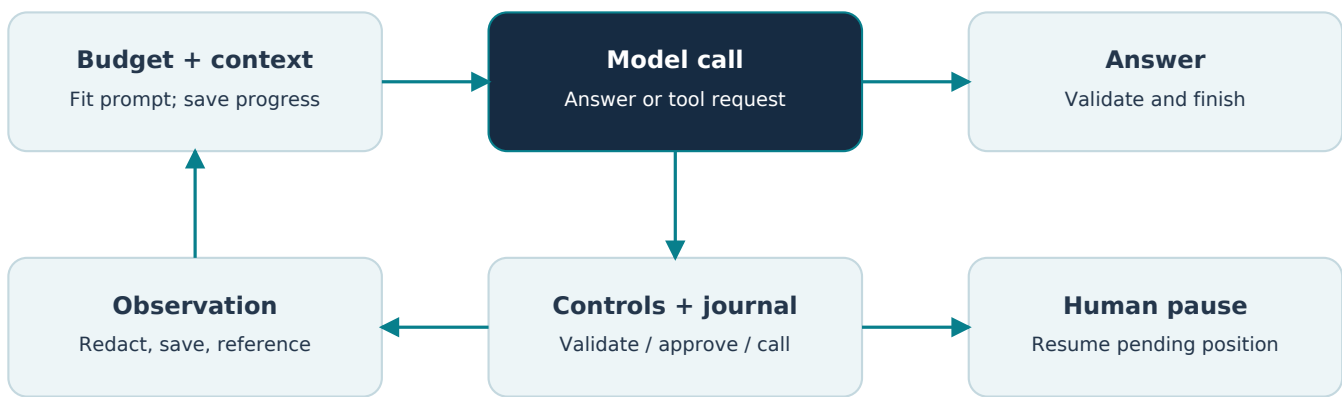
4.1 Preparing an agent pass

The runtime resolves effective effort and limits, restores any continuation, establishes an abort signal and reconstructs elapsed active time. It discovers attached MCP tools and applies the agent's explicit allow-list. It loads the configured worker provider and, for reflection, an optional judge provider.

The prompt combines operating instructions with a runtime clock, the current request, selected history, retrieved knowledge, applicable experiments and lessons, available skill descriptions, machine context and persistent-memory references. Retrieved and recalled content is labeled as reference data. The latest request controls the task; historical requests are not authorization to restart old work. A simple greeting has a shortcut that avoids agent model calls, MCP connections, notebook recall and delegation.

4.2 One turn, step by step

1. Check cancellation, remaining active time, model-turn limits and the token reserve needed for a final answer.
2. Estimate the prompt and offered tool schemas against the provider's context window. Compact and checkpoint context if required.
3. Persist progress before dispatching the model call and reserve its estimated maximum token cost.
4. Call the provider. Reconcile the charge using returned usage when available; use estimates otherwise.
5. If the response is a final answer, validate and complete the pass. If it requests tools, persist the response and current tool position.
6. Validate tool selection and arguments. Apply hooks, guardrails and any required human approval before dispatch.
7. For external agent tool calls, consult the durable journal, execute if permitted and persist the result. Apply output controls before exposing or saving evidence.
8. Append observations and references to the dialog, checkpoint progress, and ask the model what to do next.



Limits reserve tool-free synthesis; ambiguous writes stop recovery

Figure 3. Agent control loop

Tools in a model-returned batch are processed in order by the runtime. A batch is not automatically a parallel task graph. Parallel agent groups and `spawn_agents` provide explicit concurrency. This ordering also permits a human pause at a specific tool index without repeating already completed calls after approval.

Recoverable tool errors are returned as observations so the model can revise arguments or choose another allowed tool. Unknown tools and invalid calls are bounded by tool-selection recovery; they do not grant permission to invoke arbitrary capabilities. An ambiguous external write is a different category: the runtime must stop rather than reason that a blind retry is probably harmless.

4.3 Stopping with useful evidence

Budget exhaustion, the turn limit or an approaching time limit can switch execution into finalization. The finalization model call has no tools and receives saved notebook evidence. It must report completed work, missing evidence and unfinished checks. Calls requested during finalization do not execute.

The reserve makes completion more likely, but a failed provider, impossible context or insufficient remaining allowance can still prevent synthesis. The fallback preserves a readable incomplete-assessment result and note references. No mechanism guarantees an answer when the model service is unavailable.

Source basis: [S04], [S10], [S11], [S12], [S13].

5. Effort, token accounting and termination

5.1 Current presets

Effort	Analysis turns per pass	Agent token allowance	Active time	Plan steps / critiques / iterations
Light	4	30,000	90 seconds	3 / 1 / 2
Medium	12	120,000	5 minutes	5 / 1 / 3
High	24	400,000	10 minutes	8 / 2 / 6
Extra high	36	1,000,000	15 minutes	8 / 3 / 8
Max	120	5,000,000	1 hour	8 / 3 / 10

The current source sets Max to 120 turns and one hour. Older overview tables that show 40 turns and 15 minutes are stale. Previously saved agents retain explicit limits; selecting a preset again or editing Limits applies new values. Custom schema limits allow up to 200 analysis turns per pass, two hours and 50 million tokens.

Auto effort is a deterministic heuristic, not a separate planning model. It examines attached tools, whether the request exceeds 1,500 trimmed characters, and whether the pattern is plan-execute or loop. It selects light, medium or high. A short request without tools can use light; a long request with tools uses high; multi-pass planning/loop work gets at least medium. Auto applies the selected preset at execution time. Fixed effort preserves explicitly stored overrides.

5.2 Four different limits

The context window limits one model request. The token allowance limits accumulated prompt and completion usage across an agent's pattern passes. The active-time allowance limits the agent's execution duration, excluding time spent waiting for human input across continuations. The workflow step budget limits graph traversal. These limits solve different problems and do not reset one another.

An agent's accumulated token charge includes its reflection judge and runtime-delegated children. It is not a single installation-wide allowance or necessarily a total budget for every sibling node in a harness. Independent graph agents receive their own configurations and budgets. Post-run lesson generation is a separate reflection job; it should be included separately in operating-cost estimates.

The runtime calculates an overall turn guard from $(\text{maxTurns} + 1) \times \text{patternPasses}$. The pass multiplier is 1 for ReAct, maximum plan steps plus 2 for planning, 1 plus twice the critique rounds for reflection, and iterations plus 1 for looping. These are upper guards, not expected consumption. Token or time exhaustion may end the agent much earlier. The extra turn supports final synthesis.

5.3 Recovery preserves expenditure

Before a model call leaves the process, its maximum estimated cost is charged conservatively. If the worker dies before observing actual usage, recovery retains that charge. A known context rejection can release it because the request was rejected. Completed passes, child allocations, elapsed active time and used tokens survive continuation. Restarting a runner is therefore not a way to obtain a new budget for the same job.

Source basis: [S04], [S05], [S06], [S10], [S13].

6. The four built-in agent patterns

6.1 ReAct: action guided by observations

ReAct runs one tool-enabled conversation pass. The model can answer immediately, call an allowed tool, read its observation, perform another call and then answer. The harness does not expose private model reasoning; the observable record is the model's authored text, tool requests, results and runtime events.

This is the default pattern for assistants and narrow workflow steps. For example, a diagnostic agent can inspect a host, fetch a relevant notebook entry, compare the observations and report a fault. The order of tool selection is model-driven, while permissions, validation, budget and stopping behavior remain runtime-controlled.

6.2 Plan-and-execute: explicit staged work

The planning pass requests a short numbered plan without external tools. The runtime parses nonempty lines, removes common numbering/bullet prefixes and keeps at most the configured number of steps, up to eight. This is text parsing, not a general dependency-graph compiler. An empty plan fails.

The plan becomes a durable `execution_plans` record. Tasks carry IDs, order, status and saved output. Each pending task moves to in-progress, receives a tool-enabled pass, then stores its result as completed. Later tasks receive the original request, plan and prior step outputs. Saved task outputs and the summaries passed between steps are bounded to 12,000 characters each.

After the steps, a synthesis pass answers the original request using their results. Planning and synthesis prohibit external action; an eligible `load_skill` tool can still be offered in otherwise tool-less passes. The budget-limit finalization path is fully tool-free.

The API can edit pending plan tasks, and the runner obtains subsequent tasks from the durable plan. Running and completed tasks cannot be rewritten through that edit path. Resume reuses completed outputs rather than redoing the plan. A runtime loop guard also bounds task execution if a plan is changed while active.

Use this pattern where the work has meaningful milestones: inspect, compare, verify, synthesize. It introduces more model calls and repeated context than ReAct. It does not automatically create parallel plan steps or guarantee that a generated plan is optimal.

6.3 Reflection: draft, critique, revise

Reflection first creates a tool-enabled draft. A tool-less critique identifies factual gaps, unsupported claims, missing steps and clarity problems. A revision pass then has tools available to verify and improve the answer. One to three critique/revision rounds are supported.

By default the critic uses the worker model. `judgeProviderId` selects a different configured provider/model for critique. The runtime switches back to the worker for revision. Both consume the same agent budget. Choosing a second model can diversify review, but it is not independent factual verification unless relevant evidence is obtained.

Playground shows compact working status during reflection. Completed draft, critique and revision text is saved as agent activity, with agent/model labels, outside the bounded event buffer. The final answer becomes the chat result. Interrupted stages are distinguishable from completed stages. These are authored stage outputs, not hidden chain-of-thought.

6.4 Autonomous loop: progress with a completion marker

The loop pattern repeats a tool-enabled pass, carrying earlier progress into the next iteration. The prompt asks the agent to finish with its configured marker, `DONE` by default, when complete. The runtime checks for that marker at the end of the output and removes it from the returned answer. Iterations are bounded from one to ten.

If no marker appears by the iteration limit, the runtime emits `loop_limit` and performs final synthesis describing completed and unfinished work. Accumulated progress is capped to the most recent 24,000 characters; detailed evidence belongs in task notes. Token and time limits can stop the pattern before its configured iteration count.

A marker is the model's completion signal, not a verified postcondition. For consequential work, add a separate verification agent, explicit tool check, condition or human review. A loop can discover and execute next actions; it is not an unlimited autonomous process running after the job ends.

Pattern	Best fit	Main cost or limitation
ReAct	Direct assistance, tool lookups, narrow diagnostics	Model chooses sequencing; milestones are implicit
Plan-and-execute	Multi-step work with inspectable progress	Sequential steps and repeated context; plans can be incomplete
Reflection	Review, writing, analysis with verification	Additional calls; critic quality depends on model and evidence
Autonomous loop	Iterative investigation with a clear finish condition	Self-declared completion and bounded carried progress

Source basis: [S04], [S05], [S06], [S14], [S15].

7. Harness graphs and composed patterns

7.1 Graph execution semantics

A harness is stored internally as a workflow. It has a start node, nodes, resource attachments, bindings and a maximum number of traversed steps. The interpreter maintains a cursor, `last` value, per-node outputs and attempt counts. It checkpoints the cursor before starting each node. A bounded cycle can revisit nodes; its new attempt identity distinguishes new work from recovery of an in-flight attempt.

The default step budget is 100, configurable up to 500. Definitions support up to 100 nodes, 100 resources and 200 bindings. This is a bounded workflow interpreter, not a general-purpose distributed DAG engine with arbitrary user code. Results larger than 200,000 characters fail a node-output guard.

Node	Behavior
Start	Initializes execution with the request and follows its next edge
Agent	Renders a prompt and runs the configured agent pattern
Parallel	Runs selected agent cards together, waits for results and returns an array
Tool	Calls one named MCP tool with rendered arguments and configured controls

Node	Behavior
Condition	Evaluates equals, notEquals, contains, truthy or greaterThan and selects an edge
Review	Creates a durable human review and selects approval or rejection edge
Email	Sends a rendered message using workspace SMTP or installation defaults
Finish / legacy output	Renders final output and stops graph traversal

MCP, knowledge and guardrail cards are resource attachments. A knowledge card provides evidence access; it is not a standalone reasoning step. Guardrail bindings add policy to the associated execution scope. An explicit Tool node performs a predetermined action, whereas an Agent node may choose among its allowed tools.

7.2 Data movement

Templates read `{{input}}`, `{{last}}`, `{{steps.nodeId}}`, structured payload values and selected-device fields. A template consisting entirely of one reference preserves its underlying value; interpolation within a larger string converts the value to text. Rendering rejects unsafe prototype paths. Conditions operate on rendered values rather than evaluating arbitrary code.

A parallel group gives its members the group's rendered prompt. Members use separate execution keys and retain completed results for resumption. In new harnesses, up to eight agent-card references fit a group. The legacy saved-agent reference array also has its own limit; the normal designer model is the agent-card group.

Parallel groups use in-process promises in the runner handling the parent run. They are not independently leased RabbitMQ jobs. A non-human member failure aborts siblings through a shared cancellation signal; the interpreter waits for settled outcomes and raises the failure. Human pauses preserve completed members so they can be reused after input arrives.

7.3 Useful compositions

Composition	How the harness implements it	Design consideration
Sequential specialists	Researcher → Analyst → Writer, passing named step outputs	Each role can use a different provider, instructions and budget
Router	Triage agent → Condition → selected specialist	Use a constrained routing label and an explicit fallback branch
Parallel research and synthesis	Parallel group → synthesis agent	Independent contexts; synthesis must reconcile disagreements
Research and independent review	Research agent → reviewer agent	A separate node can have different permissions and evidence scope
Manager and workers	Coordinator agent with <code>spawn_agents</code>	Task allocation is model-driven; child work shares the coordinator's budget
Bounded improvement cycle	Draft → Review/Condition → revise edge	Define termination and ensure the workflow step cap bounds repetition

Composition	How the harness implements it	Design consideration
Action with approval	Agent/tool proposal → approval or Review → action/finish	Approval and observed execution outcome are distinct records
Scheduled investigator	Time trigger → agent → saved memory and optional email	Schedule deduplication prevents duplicate submissions for one slot

These are compositions of supported primitives, not additional named built-in agent patterns. Debate, voting and map/reduce-style tasks can be assembled from parallel agents and a synthesis/condition step, but the runtime does not provide a specialized consensus protocol, arbitrary dynamic graph expansion or an unbounded recursive agent swarm.

Source basis: [S04], [S06], [S16], [S17].

8. Dynamic subagents and bounded delegation

8.1 Runtime delegation

Delegation is opt-in on the coordinating agent. When enabled, `spawn_agents` accepts focused, self-contained tasks, an optional selected parent skill, light/medium/high effort and an optional subset of parent tools. A single invocation accepts up to four children. The configured total is four by default and can be raised to twelve children during the agent's execution.

Each child uses the parent's provider and operates as ReAct with a fresh dialog. It inherits operating restrictions, permitted knowledge/notebook scope, safety policies, human settings and cancellation. It can receive a narrower tool subset but cannot obtain tools absent from its parent. Children cannot recursively delegate. Their contexts contain their assigned tasks and inherited instructions, not the full parent transcript.

Children run concurrently inside the parent runner. Each is nevertheless persisted as its own run with `trigger: subagent`, parent run/node identifiers, original snapshot, token usage, status and events. The distinction matters: durable identity and traceability do not imply that each child occupies a separate broker worker slot.

8.2 Budget arithmetic

For parent remaining allowance R and N requested children, each child receives the smaller of its effort preset and $\text{floor}(0.8 \times R / N)$. Twenty percent remains reserved for parent synthesis. A share below 4,000 tokens causes the spawn request to be refused. All observed child expenditure is charged back to the parent.

Example: a parent has 100,000 tokens left and requests four light children. Each receives $\min(30,000, 20,000)$, or 20,000 tokens. The allocation leaves 20,000 for the parent before other ongoing expenditure. If only 15,000 remain, four children would receive 3,000 each, so the call is rejected; fewer children or direct work are required.

8.3 Results and shared memory

Children share the root query's temporary notebook and can find sibling notes immediately. They can also write to inherited persistent notebooks. Each final report is automatically saved under temporary [reports/](#) with a stable note ID. The parent receives a short summary, note references, status and token usage; the summary stored on a child run is bounded to 1,500 characters. Full reports are available through the saved note and answer artifact rather than a large transcript copied into the parent context.

Child IDs derive from the parent run, delegation call and child position. A recovered delegation reuses those identities, snapshots and allocated budgets. Completed children return saved results; unfinished children resume their own continuations. Stable report and observation IDs reduce duplicate notes after replay. If the parent ends before a child completes, that child is marked interrupted.

8.4 API-managed children are a separate surface

The Open Harness API also exposes child creation, delegation, termination, results and streams. Those delegate calls use an independent task budget capped at 30,000 tokens and normal workspace active-run admission. They should not be described as sharing the runtime [spawn_agents](#) budget formula. Both surfaces preserve inherited permissions and prohibit recursive delegation, but their scheduling and accounting contracts differ.

Source basis: [S04], [S06], [S18], [S21], [S22].

9. Context engineering and evidence preservation

Long tasks can exceed a model's context window even when the total job token budget is large. OpenHarness treats context size and accumulated expenditure separately. Reusing a smaller prompt does not refund tokens already spent.

9.1 Fitting a model call

The context allocator reserves the greater of 256 tokens or eight percent of the window as tokenizer/wire-format margin, then reserves output allowance. It estimates text at approximately 3.5 characters per token, accounting for message overhead and tool schemas. Usage or provider error information can refine estimates and reveal a lower effective context limit.

Compaction escalates: shorten older tool results; remove older completed turns while preserving the system instructions and current request; then reduce exchanges in the current turn if necessary. Tool calls remain paired with their observations. System instructions are not silently truncated to force a request through. If the mandatory instructions cannot fit, the runtime has to fail or produce a constrained fallback.

Recognized context-length rejections trigger bounded adaptation and retry. Malformed tool selections and certain model-response failures have separate bounded recovery paths. Authentication, transport and cancellation failures should not be described as universally retryable context errors. There is no general promise of automatic cross-provider failover.

9.2 Saving displaced context

Automatic compaction is enabled by default. Before compression, displaced material is archived into temporary context notes. Large checkpoints are split into pieces of up to 180,000 characters and their IDs are recorded. A compact excerpt and note references remain in the active dialog. They are labeled incomplete reference excerpts, not verified findings.

The agent can recover detail with [memory_read](#) and paged reads. Turning off proactive compaction does not remove context-window enforcement or authorize unlimited prompts. Context-note storage is also distinct from automatic MCP result offloading.

9.3 Tool-result offloading

Post-hook tool results can be saved in temporary [scratch/](#) notes. Above 6,000 characters, the dialog keeps an excerpt of roughly 1,500 characters plus the note reference. At most 200,000 characters per result are retained, with truncation disclosed. When offloading is disabled, large inline results are still bounded, with a 12,000-character cap on that path.

This architecture uses notebooks as an evidence store while the prompt holds a working set. It reduces repeated context load and preserves useful details across passes and agents. It does not mean all external output is retained forever: task-note TTLs, result caps, redactions and artifact limits still apply.

Source basis: [S04], [S10], [S11], [S12], [S19].

10. Memory is several systems, not one cache

10.1 Temporary task notebook

Every query receives a root task identity. All graph steps and runtime children working on that query share its MongoDB [task_notes](#). Writes are append-only, use distinct note identities and do not wait for embedding/indexing. Searches are literal and immediately available; reads support pagination. An agent write is limited to 20,000 characters, and [memory_read](#) pages can be up to 8,000 characters.

Notes contain title, kind, folder, content, creator, run/task provenance, sources and expiry. Two agents using the same title create independent notes. Notes expire seven days after creation. Reads filter expired notes immediately; a MongoDB TTL index deletes them later.

A new conversation message starts a new notebook. As a scoped exception for continuity, the run can read notebooks from the last five eligible completed top-level turns of the same conversation, within retention. The lookup includes succeeded, failed and interrupted turns. New writes belong to the new turn. Another conversation or tenant cannot gain this access by guessing a note ID.

10.2 Persistent notebooks

Persistent Markdown notes use knowledge-base document records in MongoDB and bodies in shared file storage. They have folder paths and YAML provenance, including run, agent, source and confidence/decision information where supplied. [kb_write](#) creates a new note; an agent correction can reference an earlier record instead of overwriting it. Authorized people can manage notes in the Knowledge UI.

The default write destination resolves in this order: agent-specific notebook, inherited workflow notebook, then the first plain knowledge attachment. [knowledge_base_id](#) can select another allowed

attached destination. The same notebook can be attached to several agents for shared long-term knowledge; ownership and attachment checks still apply.

Current attachment behavior is broader than an older “reference-only attachment” description: a plain knowledge attachment can supply the notebook and default feedback learning. Explicit settings take precedence. An explicit dedicated notebook retains its opt-in learning default. This guide follows the current resolver implementation.

Tool	Meaning
<code>memory_write</code>	Append intermediate work to the current task notebook
<code>memory_search</code> / <code>memory_read</code>	Find and page through current or explicitly permitted predecessor task notes
<code>memory_promote</code>	Copy a selected temporary note to an allowed persistent notebook with provenance
<code>kb_search</code> / <code>kb_read</code>	Search and read attached long-term notebooks and documents
<code>kb_write</code>	Save a durable finding, decision, feedback, note, environment observation, conversation note or lesson
<code>core_memory_read</code> / <code>core_memory_write</code>	Access concise persistent agent memory blocks where supported by the harness/API scope
<code>workspace_list</code> / <code>workspace_read</code> / <code>workspace_write</code>	Manipulate harness workspace files, distinct from knowledge notes and host files

10.3 Retrieval and additional stores

Knowledge ingestion supports Markdown/text notes and uploads such as TXT, CSV, JSON, YAML, text PDF and DOCX. Documents are indexed asynchronously. Retrieval supplies bounded passages with source references. Recent-note search combines index hits with a bounded direct scan so new notebook writes remain findable before indexing completes. This fallback is not an exhaustive semantic search of every historical document.

Weaviate provides the default hybrid search. Drivers also support Qdrant, OpenSearch, Elasticsearch and a compatible Vector Stores API. The latter delegates document processing and embedding to the external store; the other paths generally use the configured embedding provider. Store and tenant filters are preserved. A knowledge base retains its selected store; switching a populated base is not transparent migration.

Agent core memory is a separate API feature with labeled blocks, read-only enforcement, archives and import/export. It supports at most 50 blocks per agent and a 12,000-character prompt allowance. Harness workspace files provide file CRUD and search without exposing the API server's host filesystem. Neither replaces experiment records or the task notebook.

Source basis: [S11], [S19], [S20], [S21], [S26], [S27].

11. How experiments and learning are stored

11.1 What an experiment means here

An experiment is a retained account of an eligible top-level run: its task, recent conversation context, execution outcome and result. It is automatically saved when a configured notebook exists, even if the model never invokes a memory tool. It is not necessarily a controlled A/B trial, a scored benchmark or a verified scientific result.

Automatic experiment saving applies to succeeded, failed and interrupted top-level runs. Child runs, safety-evaluation runs and cancelled runs are excluded from this automatic path. The original execution records can still exist for excluded categories. An experiment starts explicitly unreviewed; completion does not establish correctness.

11.2 Storage path and record shape

The terminal run remains in `runs`. `notebookTargets` resolves workflow and agent notebook destinations and deduplicates target bases. `saveExperiments` creates one `documents` note per target knowledge base under `experiments/`. Its Markdown bytes live under a tenant-scoped storage key in `DATA_DIR/files`. The document record contains the logical folder, metadata, ownership, storage key and indexing state. The readable `experiments/` folder is metadata, not a promise of a same-named host directory.

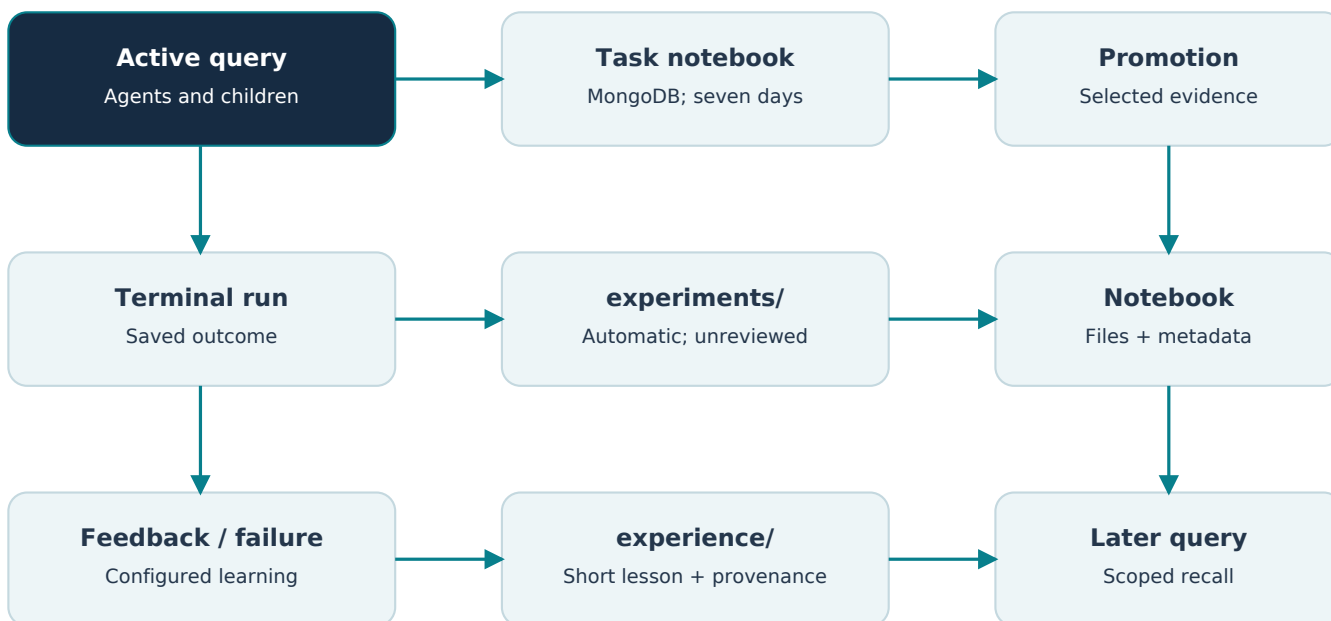


Figure 4. Memory and learning flow

The automatic body includes the last six history messages, each bounded to 2,000 characters, the task, an unreviewed outcome label and a result bounded to 32,000 characters. Metadata records `record_type`: `experiment`, `run_id`, workflow-or-agent identity, optional conversation identity, outcome, a task excerpt up to 3,000 characters and a result excerpt up to 4,000 characters. The general note builder adds provenance.

The logical note identity includes run ID and knowledge-base ID. It is hashed into a deterministic document ID, so retrying storage does not create another experiment for the same destination. Writes use separate candidate files; the deterministic document record chooses the surviving note during a

concurrent retry. The run's `experimentSavedAt` records completion of this processing. A run with no eligible notebook can also receive that processing marker, so the marker alone is not evidence that an experiment document exists.

Snapshotted input/output policies are applied to automatic experience processing. If a guardrail blocks that path, the run is marked processed without saving the blocked experiment. Original user submissions remain in the run record; notebook redaction is not retroactive deletion from all persistence.

11.3 Durable experiment saves

The worker attempts experiment saving at termination. The dispatcher also scans eligible finished runs without the saved marker, up to 50 per pass, and retries interrupted writes. This covers a worker dying after recording completion but before creating the notebook note. Vector-index availability is not a prerequisite for creating or recalling a saved experiment.

11.4 Feedback-driven lessons

A thumbs-up/down rating and optional comment are stored on the run. The API accepts the same feedback. Where learning is enabled, feedback requests a separate reflection job. Failed or interrupted top-level runs also request learning when `learnFromFailures` is enabled. A reflection receives bounded task/outcome/answer/error/tool-error/feedback evidence and requests a brief reusable lesson.

The lesson is written under `experience/`, with `lesson`, `outcome`, optional `rating`, score (+1, -1 or 0), comment, workflow identity and reflection request ID. The generation is bounded to a short answer; stored lesson text is capped at 800 characters. The provider is selected from the run's agent configuration or workspace fallback, not automatically from an agent's in-run judge setting.

Reflection state tracks pending, processing, done, skipped or failed. A processing lease permits recovery after restart, and pending work can be republished. A normal failed reflection is recorded as failed; it is not an unlimited automatic retry loop. New feedback creates a new request ID. Old lessons remain provenance records, but recall excludes superseded feedback reflections.

11.5 Recall on a later run

Recall reads up to 100 recent matching document metadata records for the same workflow-or-agent identity in the chosen notebook. With learning disabled it can still recall experiments; with learning enabled it considers both experiments and lessons. A semantic search, when available, boosts matching documents. A lexical metadata score and recency-bounded fallback remain usable if the index is delayed or unavailable.

The default selected count is three, configurable from one to ten. Recalled text identifies negative/positive feedback, failed outcomes or unreviewed experiments. It enters the prompt as reference data and emits an `experience_recalled` event naming the notes. Agents may explicitly search other shared notebook notes, but automatic recall is scoped more narrowly to the executing target.

11.6 Export and experiment discipline

`GET /api/workflows/:id/experience.jsonl` exports runs with saved experiments, feedback or lessons. Each line contains run ID, timestamp, input, output, status, error, feedback and lesson. This supports offline evaluation and future training-data preparation. It does not itself fine-tune a model, score a dataset or restore every source document and external tool state.

For comparable trials, use a fixed test input set, capture the source revision and relevant provider/model configuration, preserve tool/knowledge evidence, record the agent pattern and limits, and add external quality scores when required. Run provenance and feedback are implemented; statistical experiment

design and benchmark analysis are tasks for the experiment owner. Model weights do not change through notebook learning.

Source basis: [S08], [S19], [S20], [S28], [S29].

12. Models, tools, skills and execution context

12.1 Provider abstraction

The provider layer supports OpenAI-compatible chat endpoints, Anthropic, Gemini and Ollama. Providers define base URL, credentials, model, output settings, streaming, context window and optional embedding model. The studio can test a provider before saving and designate a workspace default. An execution can select an existing configured model through the supported API override; this does not create an unconfigured provider.

Provider adaptation normalizes messages, tool calls, text and usage for the runtime. Model limits and structured tool behavior still vary. The compatibility API accepts some legacy `temperature` and `max_tokens` fields without applying them to runtime budgets; the capability matrix explicitly documents this. The guide should therefore not imply that every accepted request field changes generation.

The runtime adds a timezone-aware clock. Agent timezone takes precedence, then schedule timezone, then UTC. Resumed execution receives a fresh time reference that supersedes earlier clock context. Response-language handling preserves the latest request's language constraints through compaction and final synthesis.

12.2 MCP capability control

Connections support Streamable HTTP and legacy SSE, with no authentication, a configured token header or OAuth. OAuth supports PKCE and server-dependent dynamic registration or a pre-registered client. Tool discovery stores schemas for inspection; execution checks actual tool availability and validates arguments before dispatch.

An agent only receives its attached allowed tools. The runtime rejects an external exposed-tool set above 120 on its discovery path; built-in tools are composed separately. Approval settings never grant a missing tool. A generic server-side code-registration endpoint or host stdio process launcher is not provided; stdio tools need a separately managed MCP bridge.

Each external agent call receives a stable idempotency key containing run, execution, pass, turn and call position. The key can help a cooperating MCP server deduplicate, but only the server can define its side-effect semantics. OpenHarness's journal does not assume that an arbitrary server honors exactly-once behavior.

12.3 Lazy skill loading and versions

A skill has a name, a short applicability description and full instructions. The agent initially sees descriptions and uses `load_skill` to obtain relevant instructions. This avoids loading every attached skill body into every prompt. Skill loads appear in the trace. Up to 20 attached skill IDs are supported per agent, and skill instructions are bounded to 32,000 characters in the core schema.

Accepted runs contain resolved skill contents. Editing a skill later does not change those snapshots. The API additionally supports SKILL.md bundles, supporting text files, validation, versions, upgrades,

rollback, import/export and discovery within harness workspace paths. Discovery is not permission to scan arbitrary server filesystem paths.

Source basis: [S04], [S06], [S12], [S21], [S30].

13. Human-in-the-loop orchestration

Agents normally have `ask_human` unless disabled. Questions, approvals and graph reviews become durable `human_requests`, surfaced in Inbox and Playground. When input is needed, the runtime saves its pending dialog, tool position, completed passes and budget state, changes the run to waiting-for-human and releases the worker. The answer requeues the continuation.

13.1 Approval policy

Approval modes are never, always and `when_risky`, with exact tool overrides. The compatibility default is never. An agent-level setting overrides the workflow default, and an exact tool rule overrides the mode. Modes apply to attached MCP tools; built-in operations require explicit entries. `ask_human` does not recursively request permission to ask.

Risk can arise from a destructive tool hint, an absent/false read-only hint or a pre-tool-hook risk score of at least 0.5. These are inputs to a permission policy, not proof that a tool is safe. A tool must still be attached and pass schema validation, hooks, guardrails and independent gateway requirements.

Authorized reviewers can approve, deny, answer or edit proposed arguments where supported. Edited arguments are validated again. If a hook changes the reviewed arguments, the proposal is denied so the agent can submit a fresh reviewable call. Approval records retain actor, decision, time, arguments and feedback; duplicate or expired decisions conflict.

13.2 Timeouts, review nodes and notifications

Default timeout is one day, configurable from one minute to seven days. Timeout actions include deny, continue and escalate. Continue can allow a workflow review to proceed with its original value under that explicit policy; it does not auto-approve a pending tool. Escalation grants an additional administrator decision period and then denies if unanswered.

A Review node has a prompt, a value and separate approval/rejection edges. A reviewer can edit the result; feedback returns with the reviewed value. This enables explicit checkpoints between analysis and action or between drafting and publication.

Optional SMTP notifications address authorized approvers and contain expiring signed links. Login and approver authorization are still required. Failed notifications are visible and retried; SMTP delivery is at least once, so a crash at the relay acknowledgement boundary can produce duplicate emails without duplicating the decision. Human waiting does not consume agent active time, but waiting runs still consume workspace admission capacity.

13.3 Machine approvals

With the gateway's Studio approval provider, required approvals are advertised in tool metadata. OpenHarness pauses before dispatch and signs a short-lived proof bound to device, tool, arguments and call ID. The gateway atomically consumes the ID before forwarding. A changed argument set, expired/unsigned proof or replay is denied. A gateway-required approval outranks an agent's never setting.

Source basis: [S09], [S13], [S23].

14. Guardrails, hooks and isolation

14.1 Additive, revised policy

Guardrail policies can be workspace defaults, workflow attachments or agent attachments. Broader defaults are additive: an agent cannot remove inherited policy. Policies are snapshotted at acceptance, so a paused run retains the revisions under which it began. Cross-workspace references are rejected. Administrators manage policy editing, deletion, defaults and evaluation; members can attach existing policies.

Checks can inspect input, output, retrieval, tool arguments and tool results. Decisions allow, block or modify/redact. Tool denials occur before dispatch. Modified evidence is used in prompts, traces and offloaded task notes. Guarded final answers are buffered before streaming. Binary tool artifacts are omitted when relevant output rails prevent safe inspection, rather than saved through an unredacted bypass.

14.2 Providers and evaluation

The built-in policy provider supports deterministic patterns and typed tool restrictions. The optional NeMo service applies the installed policy flow through its native checks API. Semantic checks use a selected configured safety model or documented fallback. The bundled deterministic baseline requires no GPU or downloaded model. It cannot guarantee protection against all prompt injection or harmful content.

Policies have per-check timeout, failure mode and durable per-policy/run latency budget. The default failure mode is closed. Check unavailability is distinguished from a content violation. Audit records retain decisions, stage, revision and latency without copying inspected payloads; their TTL is 90 days.

Templates cover bias, toxicity, hallucinations, opacity, PII and vulnerability. YAML import/export uses a constrained adapter schema rather than arbitrary Python or Colang uploads. The hallucination template assesses the supplied answer; it is not an independent external fact-checker. Opacity requests useful explanation and uncertainty, not private reasoning.

Durable evaluations use bounded safety probes with workflow-revision provenance. External tools, notebook writes, email and human review are simulated, with nested delegation disabled. The optional Garak service supplies a bounded ten-probe subset. A completed evaluation can contain failed probes. These reports are evidence, not certification or an automatic publishing gate.

14.3 Hooks and tenancy

Pre-tool webhooks can allow, deny or modify arguments; post-tool webhooks can transform output. Stop/error hooks receive terminal notifications. Hook requests are signed and time-bounded. Tool hooks default closed on failure; terminal notifications default open. Handlers are HTTP webhooks, not commands run on the harness server. Custom hook registrations are accepted but are not fired by the current runtime.

Local accounts, sessions and administrator-managed workspace teams provide the UI access model. API keys have scopes and target restrictions; embed capabilities are revocable and origin-constrained. Credentials are encrypted at rest using the installation key. Resource queries and downloads enforce tenant scope. Private-network access is governed by configured host rules, and HTTPS/proxy configuration controls secure cookies and trusted forwarding.

Source basis: [S21], [S23], [S24], [S31], [S32].

15. Resilience and recovery guarantees

OpenHarness combines at-least-once job delivery with durable execution state and conservative handling of side effects. The guarantee is recoverable progress where the stored evidence supports safe continuation. It is not universal exactly-once external execution.

15.1 Durable boundaries

MongoDB stores run state, leases, checkpoints, journal metadata, child identities and human decisions. Continuation bodies and saved tool results are JSON in shared file storage, referenced by [continuations](#) records. Replacement uses a temporary file followed by atomic rename, so readers see a complete old or new body rather than a partially written JSON file.

A continuation retains active dialog, model/tool position, completed pattern passes, prepared approvals, tokens, child count and elapsed active time. Workflow checkpoints retain cursor, prior value, completed outputs and node attempts. An agent continuation is finer-grained than a graph checkpoint; preserving both allows graph and internal pass progress to resume together.

15.2 Tool-call journal

Before an agent external call, the journal records a stable started entry. Once the result is saved, the entry becomes completed. Replay first looks for the saved result and returns it without dispatching the call again. A started entry without a result may be retried only when the call qualifies as read-only. A potentially mutating call with no observed result becomes ambiguous.

On recovery, the dispatcher examines the parent and children for ambiguous journal entries. It repairs a metadata/result boundary if the result file is present, but stops automatic recovery if the result is absent. This holds even if the request carried an idempotency key and even if workflow policy says always. Reviewing the real external state is required before a new attempt can be justified.

Explicit workflow Tool and Email nodes retain their coarser replay policy. They should not be described as having every fine-grained agent-call guarantee. SMTP acceptance followed by a crash is a classic uncertain-delivery boundary.

15.3 Policy and failure matrix

Situation	Implemented response	Remaining limitation
Broker unavailable at submission	Run remains in MongoDB; dispatcher republishes	Progress waits for broker recovery
Duplicate queue delivery	Atomic run claim prevents simultaneous ownership	Queue delivery remains at least once
Runner crash	Lease expires; dispatcher evaluates continuation and policy	Requires authoritative metadata and shared files
Completed journaled agent call	Saved result is reused	Stored result must remain readable
Read-only call without result	Can be retried	Correct tool annotations and actual server behavior matter

Situation	Implemented response	Remaining limitation
Mutating call without result	Parent/child recovery stops as ambiguous	External inspection is needed; no invented compensation
Human pause/restart	Resume saved tool index and completed passes	A separate unexpected external-action crash remains ambiguous
Context-length rejection	Compact/adapt with bounded retries	Mandatory prompt may still not fit
Provider cannot synthesize	Failed run with partial evidence/fallback	No guarantee of a complete final answer
Vector index unavailable	Task notes and bounded notebook/experiment fallback remain usable	Full semantic document recall can be degraded
Experiment save interrupted	Dispatcher retries with stable note identity	Needs notebook metadata and file storage availability
Gateway/device disconnect	Call can fail or have uncertain outcome	A lost response does not prove a machine action failed

Safe is the default workflow policy. It permits durable agent continuation or steps that can be repeated safely. Never disables automatic recovery. Always opts into graph-step replay but does not override ambiguous agent journals. [MAX_RESUMES](#) defaults to three and the schema allows zero through ten; changing deployment behavior requires actually supplying the environment value to the application.

Runner queue loss aborts active execution and exits so its supervisor can restart it. Graceful shutdown also hands recovery back to the dispatcher. Database heartbeat failure aborts the worker's active run. API dispatch ticks every three seconds when available, but this is not a strict recovery-time SLA: dependency availability, scan limits, workload and backlogs affect latency.

Source basis: [S02], [S03], [S04], [S07], [S08], [S09], [S13].

16. Scalability and capacity planning

16.1 Horizontal execution

Runner replicas consume the same queue. Each uses [WORKER_CONCURRENCY](#), default two and configurable from one to sixteen, as broker prefetch. With R runners and C concurrency, approximately $R \times C$ top-level jobs can be in service. Run, index, delete and reflection jobs share this queue and worker capacity, so an ingestion or learning backlog can compete with interactive requests.

Actual model-call concurrency can be higher than $R \times C$ because graph parallel groups and runtime-delegated children execute inside a worker. For example, four runners at concurrency two can service about eight top-level jobs; if each is in an eight-member parallel group, there can be roughly 64 active agent contexts before additional nesting. This is an illustrative bound, not a measured safe setting. A parent's child work also has memory, connection and safety-check costs.

[MAX_ACTIVE_RUNS](#) limits queued, running and human-waiting records per owner at submission. Runtime children are inserted directly and do not use the same admission path, although their run records can affect later counts. API-managed child delegations do use normal admission. Capacity planning therefore needs both top-level queue settings and internal fanout limits.

16.2 Bottlenecks and practical measurements

Resource	What grows	What to measure
Model provider	Concurrent calls, prompt/output tokens, retries	Latency percentiles, quota responses, token throughput and cost
Runner process	Dialogs, tool payloads, concurrent child contexts	RSS, CPU/event-loop delay, active jobs and abort frequency
MongoDB	Trace updates, leases, task notes and dispatch scans	Write latency, index health, storage growth and query latency
Shared filesystem	Continuations, results, knowledge bodies, artifacts	Latency, available space, cross-host visibility and rename behavior
RabbitMQ	Pending and unacknowledged jobs	Queue depth, age, confirms, redelivery and dead letters
Vector/embedding path	Indexing jobs, documents and recall queries	Index lag, embedding latency, search latency and failure rate
Gateway/tools	Open device sessions and external work	Call latency, disconnects, approval waits and target-side limits
Safety/hook services	Additional checks per model/tool action	Check latency, exhausted policy budgets and fail-closed events

An approximate steady-state top-level completion rate is effective service slots divided by mean job service time. If eight slots handle 60-second jobs, the arithmetic is about eight jobs per minute before contention and non-run jobs. A planning or reflection pattern can lengthen service time; delegation can shorten wall-clock time for independent work while increasing simultaneous resource demand.

Measure queue age and end-to-end completion separately from active model time. Human pauses release worker slots but retain admission occupancy. There is no built-in autoscaler, universal provider quota scheduler or weighted tenant-fairness guarantee established by this review.

16.3 Shared storage and infrastructure availability

All API and runner replicas must see the same files. The repository includes [compose.shared-fs.yaml](#) for a host/shared mount and documents UID 1000 write access. Independent local disks on different worker hosts would break access to uploaded documents and continuations. The storage layer is filesystem-based; an object-store adapter is not supplied by the inspected implementation.

Adding runner replicas does not make MongoDB, RabbitMQ, vector storage, the shared filesystem or the device gateway highly available. Replicated infrastructure, backup/restore, gateway session routing and load-balancer behavior require deployment design and testing. Device WebSocket sessions are held in gateway process memory; simply placing gateways behind a round-robin load balancer does not demonstrate cross-instance routing.

The API also runs recovery and scheduling loops. Durable claims and deterministic keys provide coordination for specific jobs, but multi-API operation and fairness should be validated under the intended traffic. The default stack is a practical self-hosted installation, not an asserted unlimited distributed control plane.

Source basis: [S02], [S03], [S07], [S08], [S18], [S25], [S33].

17. Machines and fleet-scale agent orchestration

17.1 Machine control as MCP

Linux, Windows and Chrome connectors dial outward to the gateway, allowing operation without public device IPs. The gateway presents authenticated per-device MCP endpoints and fleet discovery. Tool allow-lists exist at both gateway and connector levels, alongside connector-specific restrictions such as path boundaries, command policy, timeouts and output caps.

Selecting a machine for a normal run replaces prior device bindings while retaining other allowed non-device MCP tools. Agents receive explicit machine context. Machine policies and approval requirements still apply; selecting a device is not an instruction to bypass its allow-list.

The gateway keeps enrollment metadata and audit records in its own MongoDB database. Live sockets remain process state. TLS, trusted proxy settings and outbound-network configuration are deployment responsibilities. Native Linux, Docker and browser/Windows installation assets are included; GPU fleet installation additionally supports generated Kubernetes DaemonSet YAML.

17.2 Durable cluster monitoring

A cluster provides shared enrollment, bounded identity capacity, a monitoring agent, interval and concurrency settings. Default monitoring is a five-minute interval with concurrency four. A cycle snapshots online membership, persists cursor/active run IDs/outcome counts and queues node-scoped runs in waves. Idempotency keys protect submissions across coordinator restart.

Each monitoring run replaces all external MCP connections with the chosen node. It disables internal delegation and caps analysis to eight turns and 16,000 tokens, or the agent's lower configured values. The fleet coordinator, rather than a model-created child tree, controls fanout. Waiting-for-human runs continue to occupy that cycle's concurrency.

Cycles for one cluster do not overlap. If a cycle takes longer than the configured interval, the next cycle starts after completion plus the interval. Pausing prevents further dispatch; already accepted runs can be cancelled separately. Offline nodes are counted, and nodes that disappear before dispatch can be skipped.

For N nodes, monitor concurrency C and average node duration T , a first approximation is $\text{ceil}(N / C) \times T$, before queue/provider contention. For 2,000 nodes at four concurrent 30-second tasks, that is 15,000 seconds, or about 4 hours 10 minutes. A five-minute interval is therefore not a guarantee of a five-minute complete fleet scan. The repository documents bounded-queue and 2,000-identity storage tests, not a hardware throughput benchmark for 2,000 GPUs.

17.3 Diagnostics and controlled remediation

Cluster capabilities are typed around system information, GPU inspection and permitted GPU remediation. Host diagnostics require explicit host access and installed host utilities; missing tools return diagnostic errors. The orchestrator remains unprivileged while a node connector may have explicitly configured root/host namespace access.

Diagnostics are the default. Disruptive actions require configured human approval or automatic-within-limits mode, gateway action policy, node action allow-list and cooldown. The external scheduler/operator must drain the node and create a recent protected drain marker. The integration

does not implement scheduler-specific drain/rejoin itself, and cluster tools do not let the agent create that authorization marker.

Before disruptive work the connector verifies no GPU compute jobs are active; uncertainty fails closed. Gateway cooldown limits disruptive requests per cluster, while the node persists its own reservation. A single-GPU reset does not silently widen to other GPUs. A lost reboot/reset response remains uncertain and must be checked against machine state. Tests simulate remediation commands; this guide makes no claim of real hardware resets performed during documentation work.

Source basis: [S25], [S33], [S34].

18. Studio, APIs, schedules and integrations

18.1 Studio workflows

The harness library and canvas manage agent cards and attachments with popup configuration, drag-and-drop resources, branching, parallel groups and YAML. Starter recipes provide concrete starting shapes. Agent settings group general behavior, reasoning, knowledge and safety/human controls.

Playground supports harness-specific saved conversations, live text, trace, Memory, history, machine selection and pending human requests. Reflection activity is inspectable without repeatedly replacing the visible answer with intermediate drafts. Knowledge pages edit notebook notes and inspect indexed passages; Skills manage reusable instructions; Guardrails manage policies and evaluations; Machines & clusters manage enrollment and monitoring. Settings include providers, workspace/team and SMTP. The API explorer makes supported operations and request/response behavior inspectable.

18.2 Conversations and sessions

Conversation/session turns are durable and serialized: another message conflicts while the current turn remains active. Sessions support history, fork, pause/resume/end, streaming and authenticated WebSockets. History retains the latest 100 messages and the model prompt uses the latest 20, subject to context compaction. Failed/cancelled turns are not simply appended as successful conversation memory; saved run evidence remains separately available.

Session history, task-note continuity and persistent notebooks are separate channels of continuity. A follow-up can draw on earlier conversation text and eligible recent task notes while writing new evidence under a new task identity. Ending a session prevents it from being resumed as an active session.

18.3 External execution and scheduled entry

Applications can submit and inspect runs, stream progress, cancel execution, provide human input and download results/artifacts. Authenticated JSON webhooks trigger harnesses. Revocable iframe embeds have origin restrictions. Outgoing signed webhooks expose lifecycle and selected human/safety events with durable delivery tracking.

Schedules support intervals and wall-clock daily/weekly operation with timezone handling. Persisted deterministic schedule keys deduplicate an accepted slot across restart and dispatcher races. A scheduled run uses an accepted snapshot like other runs. Schedule deduplication is not a promise that an external tool action is exactly once.

Email nodes render recipient, subject and body from graph scope and use workspace SMTP settings over installation defaults. Email is an explicit graph action; it does not make every agent an unrestricted mail sender unless a separate mail capability is also attached.

18.4 Open Harness API coverage

The adapter exposes harness/agent lifecycle, execution, sessions, MCP, skills, memory, files, subagents, hooks, planning, diagnostics and read-only conformance reporting. Harness-scoped APIs are the preferred model; compatibility aliases retain older workflow/agent behavior. OAF agent manifests support import/export/clone with configured reference resolution and warnings for unsupported fields.

Generated support tables and capability endpoints state limitations. Remote IDE/process launching, arbitrary custom-code tool registration and direct host stdio execution are outside the adapter. Conformance status is partial evidence, not certification. A repository client supports JSON, multipart, raw files and streams without additional client dependencies.

Execution SSE supports progress, text, tool events, errors and terminal status, with [Last-Event-ID](#) replay. Terminal streams remain available for one hour; stored results remain readable afterward. A synchronous session that pauses for human input returns an input-required response with the execution identity.

Source basis: [S01], [S16], [S21], [S22], [S35], [S36].

19. Storage, retention and observability reference

Data	Authoritative location	Scope and retention
Resources and users	MongoDB resource/account collections	Tenant-owned; application lifecycle, no universal TTL asserted
Run snapshot/result/checkpoint	runs	Run/tenant; retained history; events bounded to latest 500
Live partial text	Run buffer	Bounded to latest 32,000 characters; transient, replaced by terminal result
Agent pass continuation	continuations metadata + shared files	Run/execution key; atomically replaced body
External-call journal	tool_journal + continuation result files	Stable call identity; records started/completed boundary
Human decisions	human_requests	Run/approver scope, explicit expiry and lifecycle
Plan/tasks	execution_plans	Execution/node key, task status/output and revision
Reflection stage text	agent_activity + continuation files	Full completed authored stages outside bounded events
Task notebook	task_notes	Root task and tenant; seven-day TTL
Persistent notes/documents	documents + shared file bodies	Knowledge-base scope; experiments and lessons have provenance
Search index	Configured vector store	Derived retrieval data; backend-dependent indexing lifecycle
Agent core/archive memory	Dedicated memory records and archive search storage	Agent/harness scope; distinct from notebook experiments
Workspace files	Harness file records + storage interface	Harness scope; distinct from server filesystem
Artifacts	Artifact records + shared file bytes	Execution scope; 50 per run, 5 MiB each
Harness events	harness_events	Tenant event feed; seven-day TTL
Webhook deliveries	webhook_deliveries	Delivery tracking; 30-day TTL
Guardrail audit/budget records	guardrail_audit , guardrail_budgets	Audit 90 days; budget records 30 days
Gateway audit	Gateway tool_calls	Separate gateway database; documented default 90 days, configurable

Data	Authoritative location	Scope and retention
Cluster cycle state	<code>cluster_cycles</code> , monitor records	Membership/cursor/active IDs/counts with coordinator lease

19.1 Reading an agent execution

A useful review starts with the original request, accepted configuration and terminal status. Follow graph node events to locate the responsible agent; inspect its model/tool events, stage outputs, task notes and artifacts. For a delegated task, open the child run and then the full report note. For a remembered conclusion, follow experiment/lesson provenance back to its source run and feedback.

The trace buffer is bounded, so it is not a complete permanent event-sourcing log. Full saved reflection stages, task notes, plans, artifacts and experiment documents carry additional evidence. Some raw material is deliberately capped or expires. Export important evaluation evidence while it remains available and distinguish observed facts from model-written conclusions.

Artifacts capture final answers, embedded MCP resources and qualifying successful machine file reads/writes. Truncated reads and append outputs are labeled as partial. Arbitrary tool URLs and server paths are not fetched just because a tool mentions them. Downloads enforce execution/token scope and attachment headers. Hook/safety redaction is not bypassed to obtain an unredacted artifact.

19.2 Backups and growth

Backups must cover MongoDB, broker state, vector storage and shared files consistently, plus the installation encryption key and configuration. The key is necessary to decrypt stored credentials. Backing up only the vector database cannot restore experiment metadata or continuation state; copying only Markdown bodies loses ownership and linkage.

Task-note and audit TTLs bound some growth. The inspected code does not establish one universal retention/garbage-collection policy for all runs, continuations, artifacts and persistent notes. Plan storage monitoring and retention for those durable classes explicitly. The single-host operational guide describes quiesced, consistent backups; online replicated disaster recovery needs deployment-specific procedures.

Source basis: [S02], [S08], [S09], [S11], [S15], [S19], [S27], [S32], [S33], [S36].

20. Worked agentic scenarios

The following examples are designs using implemented primitives. They illustrate how the pieces interact; they are not claims about installed user harnesses or measured outcomes.

20.1 Research with delegated evidence and reflection

Configure a coordinator with a persistent notebook, selected research MCP tools, a relevant skill, high effort and delegation enabled. Give it a specific question and evidence standard. It loads the skill and starts three independent children: source discovery, numerical validation and contrary-evidence review. Each child receives a bounded share of the coordinator's remaining allowance.

Children write source-backed findings to the shared task notebook and optionally durable findings to the attached notebook. Their final reports are automatically saved, and the coordinator gets summaries with note IDs. It reads the relevant report sections and synthesizes a draft. A subsequent reflection node critiques and revises the synthesis, potentially using a different judge provider.

Because the reviewer is a separate graph agent, its budget is independent of the coordinator's budget. The coordinator's children are charged to the coordinator. At completion, the top-level run creates unreviewed experiment notes in deduplicated eligible notebooks. A later negative rating can generate an experience lesson, while the original reports remain temporary unless promoted or explicitly saved persistently.

If the runner dies after two children finish, their IDs and saved reports are reused. The unfinished child resumes its own continuation if its external calls are unambiguous. If a child issued a mutating call and the result was lost, automatic recovery stops for the parent rather than launching a replacement child that repeats the action.

20.2 Investigate, approve and verify a machine change

Use a machine-scoped diagnostic agent with read tools and a notebook. Require approval for the proposed write/remediation tool. The agent inspects state, records observations and proposes an exact action. The run pauses with a saved tool index and releases its worker. The authorized operator reviews the arguments and approves or denies.

On approval, the continuation executes the validated action. A verification agent or explicit read tool checks the postcondition. Save the observed before/after state and link it to the decision. An approval means the action was authorized; only the tool result and verification show whether it succeeded.

If the external system accepts the change but the response is lost, a missing result cannot be interpreted as failure. Inspect the system before starting a new attempt. For GPU remediation, the cluster's drain-marker, no-active-compute, allow-list and cooldown requirements also apply.

20.3 Continuous quality improvement

Run a fixed collection of representative requests against two harness configurations, for example ReAct and plan-and-execute. Preserve each configuration identity, source revision, model selection, evidence and limits. Let both save experiment notes; attach human ratings and a separate rubric or automated external scorer.

Export experience JSONL for analysis of answer quality, failure types and feedback. Retrieve additional token/trace/configuration evidence from run records where needed; JSONL is not the complete execution bundle. Review proposed lessons before treating them as reliable reusable knowledge. Keep evaluation datasets separate from raw unreviewed memories if accidental recall would contaminate the comparison.

This process uses the harness's execution provenance and memory infrastructure. The repository does not supply a general experiment dashboard with significance testing, automated model training or a guarantee of a reproducible external environment.

Source basis: [S04], [S18], [S23], [S25], [S28], [S29].

21. Verification evidence and practical limits

21.1 Repository verification surfaces

The project defines TypeScript type checks, unit tests, browser tests and real-stack integration tests. Relevant suites cover context budgets, tool selection, final answers, notebooks, subagents, experience, human input, reliability, protocol domains, guardrails, machines and clusters. The isolated test Compose project contains deterministic model/MCP fixtures and fault-injection assets.

The current project instructions keep hosted CI disabled. Checks are run locally; a completed pull request is merged without adding a hosted-CI gate. Older documents that describe CI as actively running should not be read as the current operational configuration. This guide was created through that documentation workflow and does not alter runtime execution, authentication, storage behavior or deployment services.

Test source is evidence of intended coverage, not proof that all tests ran successfully in every installation. Deterministic fixtures can demonstrate state transitions, policy boundaries and recovery mechanics. They cannot establish live model answer quality, every MCP provider's behavior, sustained-load throughput or actual GPU recovery success.

21.2 Important limits to preserve in technical claims

- Four built-in agent patterns are implemented. Other arrangements are compositions, not hidden built-in swarm algorithms.
- Agent and runtime-child budgets are bounded. A graph with several agents does not have one automatic shared token ceiling.
- Job delivery and notifications can repeat. External exactly-once execution is not guaranteed.
- Configuration snapshots preserve agents, skills and policies, but do not freeze all provider records, tools or knowledge.
- Task notes expire and traces are bounded. Durable evidence must be intentionally retained where required.
- Experiments are initially unreviewed. Lessons improve prompts and do not update model weights.
- Recent-note and experiment fallback improve degraded retrieval; they do not replicate a complete unavailable vector index.
- Scaling runners increases execution capacity, while databases, storage, providers and gateways remain independent constraints.
- Safety checks, model critique and human approval serve different purposes. None alone proves a final answer or action is correct.

21.3 How to evaluate a deployment

Select representative tasks and measure completion quality, provider latency, total tokens, queue age, memory growth and recovery outcome. Exercise runner interruption and broker outage only in the isolated test stack. Include a human pause/resume case, a completed-call replay case, an ambiguous-write case, a vector outage case and an experiment-save interruption case.

For a multi-host deployment, additionally validate shared-file visibility and atomic replacement, dependency failover, lease behavior, device session routing and backup restore. Set workload-specific targets for successful answers and time to recovery. This is proposed validation work, not a claim that the stock stack ships with those measured service levels.

Source basis: [S01], [S02], [S03], [S37].

22. Source register

All source references below point to the reviewed commit. They provide an audit trail for the descriptions, numeric limits and implementation distinctions in this document. Related files can be reached from the same repository revision. Existing documentation is useful context; the executable source was used to resolve observed discrepancies.

- [S01] **Product overview and repository configuration.** [README.md](#); [package.json](#).
- [S02] **Operational deployment and shared storage.** [docs/operations.md](#); [compose.shared-fs.yaml](#).
- [S03] **Configuration defaults and queue semantics.** [packages/core/src/config.ts](#); [packages/core/src/queue.ts](#).
- [S04] **Agent loop, patterns and graph interpreter.** [packages/core/src/runtime.ts](#).
- [S05] **Effort presets and selection.** [packages/core/src/patterns.ts](#).
- [S06] **Agent, workflow and resource schemas.** [packages/core/src/schema.ts](#).
- [S07] **Runner ownership and terminal handling.** [apps/runner/src/worker.ts](#).
- [S08] **Submission, snapshots, dispatch and recovery.** [packages/core/src/runs.ts](#); [apps/api/src/server.ts](#).
- [S09] **Human requests and file-backed continuations.** [packages/core/src/human.ts](#).
- [S10] **Context estimation and compaction.** [packages/core/src/context.ts](#).
- [S11] **Temporary task-note implementation.** [packages/core/src/memory.ts](#); [docs/task-memory.md](#).
- [S12] **Provider and tool validation.** [packages/core/src/llm.ts](#); [packages/core/src/toolValidation.ts](#).
- [S13] **Budget and recovery behavior guide.** [docs/execution-recovery.md](#); [packages/core/src/executionRecovery.ts](#).
- [S14] **Durable plans and editable pending tasks.** [packages/core/src/executionPlans.ts](#); [apps/api/src/openharness/planning.ts](#).
- [S15] **Saved authored agent activity.** [packages/core/src/agentActivity.ts](#).
- [S16] **Starter recipes and graph resource compilation.** [packages/core/src/starters.ts](#); [packages/core/src/workflow.ts](#).
- [S17] **Template and condition semantics.** [packages/core/src/templates.ts](#).
- [S18] **Runtime delegation and child budgets.** [packages/core/src/subagents.ts](#); [docs/subagents.md](#).
- [S19] **Persistent notebooks and file storage.** [packages/core/src/workspace.ts](#); [packages/core/src/storage.ts](#).
- [S20] **Notebook selection and inheritance.** [packages/core/src/notebooks.ts](#).
- [S21] **Open Harness API coverage and limits.** [docs/openharness-api.md](#); [docs/openharness-support.md](#).
- [S22] **API-managed subagents.** [apps/api/src/openharness/subagents.ts](#).
- [S23] **Human workflow, approvals and artifacts.** [docs/human-input.md](#); [packages/core/src/artifacts.ts](#).
- [S24] **Guardrail stages, configuration and evaluation.** [docs/guardrails.md](#); [packages/core/src/guardrails.ts](#).
- [S25] **GPU fleet enrollment, monitoring and remediation.** [docs/gpu-clusters.md](#); [packages/core/src/clusterMonitor.ts](#).
- [S26] **Vector-store drivers and ingestion.** [docs/vector-stores.md](#); [packages/core/src/knowledge.ts](#).

- [S27] **Persistent agent memory and workspace files.** [packages/core/src/agentMemory.ts](#);
[packages/core/src/harnessFiles.ts](#).
- [S28] **Experiment saving, lesson generation and recall.** [packages/core/src/experience.ts](#); [docs/experience.md](#).
- [S29] **Experience JSONL export and feedback routes.** [apps/api/src/app.ts](#).
- [S30] **MCP and skill version implementation.** [packages/core/src/mcp.ts](#); [packages/core/src/skillVersions.ts](#).
- [S31] **Lifecycle hooks and failure modes.** [packages/core/src/hooks.ts](#).
- [S32] **Indexes, retention and application security.** [packages/core/src/db.ts](#); [packages/core/src/security.ts](#).
- [S33] **Gateway design and connector boundaries.** [docs/ARCHITECTURE.md](#); [docs/SECURITY.md](#).
- [S34] **Cluster and machine installation assets.** [packages/core/src/clusterInstall.ts](#); [packages/core/src/machineInstall.ts](#).
- [S35] **Conversations, schedules and API sessions.** [packages/core/src/conversations.ts](#); [packages/core/src/schedule.ts](#);
[apps/api/src/openharness/sessions.ts](#).
- [S36] **Harness events and streaming API.** [packages/core/src/harnessEvents.ts](#); [apps/api/src/openharness/events.ts](#).
- [S37] **Project instructions and local test evidence.** [AGENTS.md](#); [tests/integration/reliability.test.ts](#);
[tests/integration/experience.test.ts](#); [tests/integration/subagents.test.ts](#); [tests/compose.test.yaml](#).

Terminology

Agent: model configuration plus instructions, capabilities, memory bindings, controls and execution limits. Pass: one bounded model/tool conversation. Pattern: arrangement of passes within one agent. Harness/workflow: graph controlling nodes and resource bindings. Run/execution: persisted invocation with state, snapshot and output. Continuation: enough saved progress to resume an interrupted or paused execution. Task notebook: temporary query-scoped evidence. Experiment: retained unreviewed record of an eligible run. Lesson: feedback/failure-derived prompt guidance. Artifact: downloadable execution output. MCP: the protocol for external agent capabilities. Lease: time-bounded worker ownership recorded in MongoDB.